CampusEats — Services, Contracts & Schema

CS 543 Web Services · Assignment 2

1. Capabilities

Every distinct job the CampusEats system does, drawn from the Assignment 1 brief:

1. **Register and authenticate users** — students, vendors, and delivery agents create accounts and log in.
2. **Manage a vendor's menu** — a vendor adds, updates, or removes menu items and marks them in/out of stock.
3. **Browse the catalogue** — a student searches and filters vendors and menu items.
4. **Place an order** — a student turns a cart into a confirmed order against a vendor.
5. **Process payment** — an order is charged (and, when needed, refunded).
6. **Track and update order status** — an order moves through placed → preparing → ready → delivered/completed.
7. **Assign and manage delivery** — a delivery agent is matched to an order and its handoff is tracked.
8. **Rate and review** — a student rates a completed order and its vendor.
9. **Notify users** — order and delivery status changes are pushed to the student.

2. Services

(See `services.drawio` / `services.png` for the diagram this section summarizes — boxes, data ownership, and labelled cross-service calls.)

Service	Owns (data)	Responsible team member
User Service	Users, Credentials, Addresses, Roles	Member A
Catalogue Service	Vendors, MenuItems, Categories, Availability	Member B
Order Service *(central)*	Orders, OrderItems, OrderStatusHistory	Member C
Payment Service	Payments, Transactions, Refunds	Member D
Delivery Service	Deliveries, DeliveryAgents, Assignments	Member E
Review Service	Reviews, Ratings	Member F

Replace the placeholder member names above with your actual teammates before submitting.

No two services share a table; every cross-service need is satisfied by calling the owning service's contract, never by reading its data directly.

3. Contracts

Each table lists only the operation, its inputs/outputs, and its errors — never table names, id formats, or SQL. Order Service and Catalogue Service are the two main services (3 operations each); the rest expose at least one.

3.1 Order Service *(main)*

Operation	Input	Output	Errors
`placeOrder`	userId, vendorId, list of {itemId, quantity, customizations}, fulfillment preference (delivery address or pickup), paymentMethodToken	orderConfirmation: {orderId, status, estimatedReadyTime, totalAmount}	`ItemUnavailable`, `VendorClosed`, `InvalidAddress`, `PaymentDeclined`, `EmptyCart`, `PriceMismatch`
`getOrderStatus`	orderId, requestingUserId	orderStatus: {status, statusHistory, items, totalAmount}	`OrderNotFound`, `NotAuthorized`
`cancelOrder`	orderId, requestingUserId	cancellationConfirmation: {refundInitiated}	`OrderNotFound`, `OrderNotCancellable`, `NotAuthorized`
`updateOrderStatus` *(internal — callable only by Delivery/Catalogue on their own leg)*	orderId, newStatus, sourceService	acknowledgement	`OrderNotFound`, `InvalidStatusTransition`

3.2 Catalogue Service *(main)*

Operation	Input	Output	Errors
`browseMenu`	vendorId, optional filters (category, search text, price range)	menuList: {items: [{itemId, name, price, inStock, category}]}	`VendorNotFound`
`checkItemAvailability`	vendorId, itemId, quantity	availability: {available, currentPrice}	`ItemNotFound`, `VendorClosed`
`addMenuItem`	vendorId, requestingUserId, itemDetails (name, price, category)	itemConfirmation: {itemId}	`NotAuthorized`, `InvalidItemDetails`
`updateItemAvailability`	vendorId, itemId, requestingUserId, inStock	acknowledgement	`ItemNotFound`, `NotAuthorized`

3.3 User Service

Operation	Input	Output	Errors
`registerUser`	name, email, password, role	userConfirmation: {userId}	`EmailAlreadyRegistered`, `InvalidDetails`
`authenticateUser`	email, password	sessionToken	`InvalidCredentials`
`getDeliveryAddress`	userId, requestingServiceToken	address: {line1, city, pincode}	`UserNotFound`, `NoAddressOnFile`

3.4 Payment Service

Operation	Input	Output	Errors
`authorizePayment`	userId, orderId, amount, paymentMethodToken	paymentReceipt: {transactionRef, status}	`PaymentDeclined`, `InvalidPaymentMethod`
`refundPayment`	transactionRef, amount	refundConfirmation: {status}	`TransactionNotFound`, `RefundNotAllowed`

3.5 Delivery Service

Operation	Input	Output	Errors
`requestDelivery`	orderId, pickupLocation, dropLocation	deliveryConfirmation: {deliveryId, agentAssigned, eta}	`NoAgentsAvailable`, `InvalidLocation`
`getDeliveryStatus`	deliveryId	deliveryStatus: {status, eta}	`DeliveryNotFound`

3.6 Review Service

Operation	Input	Output	Errors
`submitReview`	userId, orderId, rating, comment	reviewConfirmation: {reviewId}	`OrderNotCompleted`, `ReviewAlreadyExists`, `NotAuthorized`
`getVendorRating`	vendorId	ratingSummary: {average, count}	`VendorNotFound`

4. Central Operation — `placeOrder`

`placeOrder` lives on the **Order Service**; it is the one call that turns a student's cart into a confirmed order, and it is where the system's cross-service orchestration is concentrated.

Inputs - `userId` — the ordering student (from an authenticated session) - `vendorId` — the vendor the order is placed against - `items` — a list of `{itemId, quantity, customizations}` - `fulfillment` — either a delivery address reference or a pickup flag - `paymentMethodToken` — a pre-tokenized payment method (never a raw card number)

Outputs - `orderId` — the identifier for the new order - `status` — `PLACED` - `estimatedReadyTime` - `totalAmount` — itemized as subtotal, tax, and delivery fee (if applicable) - `paymentReceiptReference`

Error cases - `EmptyCart` — no items submitted - `VendorNotFound` / `VendorClosed` — vendor doesn't exist or is outside operating hours - `ItemUnavailable` — one or more items are out of stock - `PriceMismatch` — the client's cached price differs from the vendor's current price (stale cart) - `InvalidAddress` — delivery requested but the address is outside the vendor's service radius - `PaymentDeclined` — payment authorization failed - `NoDeliveryAgentAvailable` — delivery was requested but no agent could be assigned; the order is still placed and flagged for delayed dispatch or pickup fallback rather than failing outright - `NotAuthenticated` — the session token is missing or invalid

Internal details `placeOrder` must hide from every caller - The exact sequence and protocol used to call Catalogue Service (`checkItemAvailability`), Payment Service (`authorizePayment`), and Delivery Service (`requestDelivery`) — whether these run synchronously in-request or via an internal event/saga pipeline. - The compensation logic when a later step fails after an earlier one succeeded (e.g., payment authorized but delivery assignment fails triggers an internal refund) — callers only ever see one final outcome, never the intermediate state. - Idempotency-key handling that prevents a retried/duplicated submission from creating two orders. - The tax and delivery-fee calculation formula and where its configuration is stored. - Internal id-generation

scheme, transaction boundaries, and the locking strategy used to safely decrement item stock under concurrent orders. - Which of the Order Service's own tables (`Orders`, `OrderItems`, `OrderStatusHistory`) are written, and in what order.

5. Service Validation

Each service checked against the five properties of a Service: **reachable**, **self-contained**, **has a contract**, **independent**, **loosely coupled**.

"Partial" = the property technically holds today but is fragile; see the fix noted alongside it.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes / fix
User Service	Yes	Yes	Yes	Yes	Yes	No issues — never calls another service.
Catalogue Service	Yes	Yes	Yes	Yes	Partial *	`verifyVendorRole` calls User Service synchronously on every menu edit, coupling menu management's runtime availability to User Service's uptime. **Fix:** accept a pre-validated role claim inside a signed auth token (e.g., JWT) instead of calling User Service per request.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes / fix
Order Service	Yes	Yes	Yes	Yes	Partial *	`placeOrder` calls three other services synchronously in one request, so a slow/unavailable dependency directly slows or fails order placement. **Fix:** keep item/payment checks synchronous (they gate success/failure) but move delivery assignment to an asynchronous event after the order is confirmed.
Payment Service	Yes	Yes	Yes	Yes	Yes	Called by Order Service only; never calls another service itself.

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Notes / fix
Delivery Service	Yes	Yes	Yes	Yes	Partial *	Calls back into Order Service (`updateOrderStatus`) on every status change, creating a request cycle (Order → Delivery → Order). **Fix:** have Delivery Service publish a `DeliveryStatusChanged` event to a queue/topic that Order Service subscribes to, removing its need to know Order Service's contract directly.
Review Service	Yes	Yes	Yes	Yes	Yes	Calls Order Service once, to confirm the order is completed, before accepting a review — a normal, minimal dependency.